Agentic BI Analyst

Governed natural-language analytics and a multi-agent cross-business review for banking: installation, results and a full build walkthrough



LangGraph · Claude · DuckDB · Streamlit

Ahmad Zulfikar Rafsanjani · github.com/vickarzz/agentic-bi-analyst

Why: SQL-writing chatbots aren't safe in a bank

Typical text-to-SQL chatbot	This project
Same question, different numbers	One governed semantic layer: every metric defined once, with an owner
Hallucinated joins and double counting	Deterministic compiler; multi-fact questions use CTEs (no fan-out)
Personal data leaks (names, phones)	Three guardrail layers plus small-cell suppression
No way to prove answers are right	Golden-question evals against independent SQL, run in CI
Narratives invent figures	Every number in the summary is computed from the result

The project at a glance

15

governed metrics

7

specialist agents running in parallel

70

automated tests

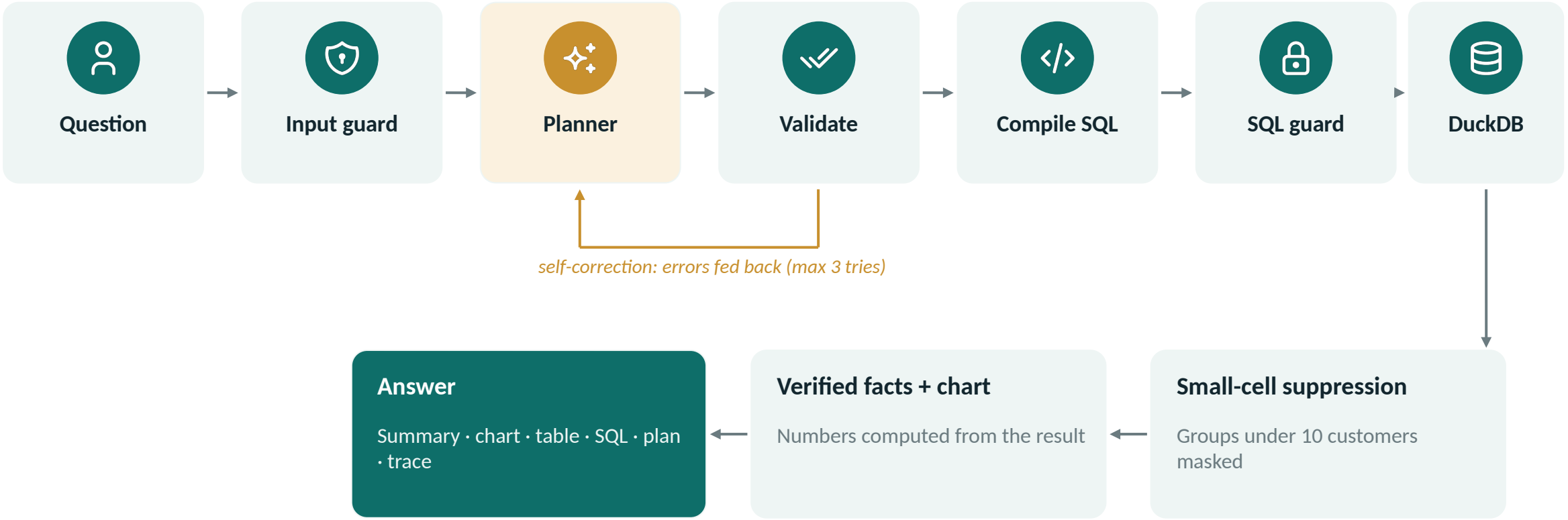
100%

golden-question accuracy (28 cases)

What it does: a manager types a question ("Which region has the highest NPF ratio?"). A LangGraph agent turns it into a structured plan that may only use governed metrics. A compiler writes safe SQL, DuckDB runs it, and the app shows the answer, a chart, the SQL, the plan and an audit trace. A second mode runs a team of agents that reviews the whole bank and tests how each business line affects the others.

Data: "Wadi Bank", a fictional Saudi retail bank: 20,000 customers, ~560k transactions, ~5.4k Murabaha/Ijarah contracts, ~174k remittances. 100% synthetic.

Architecture: AI decides what, governed code decides how



Blocked, unclear or unsafe requests exit early with a refusal or a clarifying question.

Tech stack and why each piece was chosen

Python 3.11+

One language for data generation, agent, UI and tests

DuckDB

In-process analytical SQL; no server; patterns transfer to Snowflake, Oracle, Impala

LangGraph

The agent as an explicit, testable state machine with bounded retries

Claude (Anthropic SDK)

Forced tool use with an enum schema returns reliable structured plans

Streamlit + Plotly

A usable chat UI in pure Python; free hosting on Streamlit Cloud

pytest + GitHub Actions

Tests and golden-question evals gate every push

Part 1 · Installation

From the zip file to a running app in about five minutes

Before you start

Python 3.11 or newer

python.org/downloads. On Windows, tick "Add python.exe to PATH" during setup.

```
Check: python --version
```

Git

git-scm.com. Needed to push the project to GitHub.

```
Check: git --version
```

Anthropic API key (optional)

Only for the Claude planner. Without it the app runs fully offline.

```
console.anthropic.com
```

Time needed: ~5 minutes to install, ~5 seconds to generate data. Disk: ~150 MB including libraries.

Tip: use VS Code with its built-in terminal (Terminal → New Terminal) so you can edit and run in one window.

Install in five steps

1

Unzip and open a terminal

Extract agentic-bi-analyst-v1.zip, then open a terminal inside the agentic-bi-analyst folder.

```
cd agentic-bi-analyst
```

2

Create a virtual environment

Keeps the project's libraries separate from the rest of your machine.

```
python -m venv .venv
```

3

Activate it

Your prompt will start with (.venv) when it worked.

```
Windows:  .venv\Scripts\activate
Mac/Linux: source .venv/bin/activate
```

4

Install libraries

DuckDB, pandas, LangGraph, Anthropic SDK, Streamlit, Plotly, pytest.

```
pip install -r requirements.txt
```

5

Generate the synthetic bank

Builds data/wadi_bank.duckdb with ~900k rows in about 5 seconds.

```
python scripts/generate_data.py
```

Three ways to run it

Chat app

```
streamlit run
app/streamlit_app.py
```

Opens http://localhost:8501. Click a sample question in the sidebar or type your own. Stop with Ctrl + C.

Command line

```
python -m bi_agent.cli \
  --show-sql \
  "NPF ratio by segment"

python -m bi_agent.cli
```

One question, or interactive mode with follow-ups ("now by region"). Add --trace to see each agent step.

Quality checks

```
python -m pytest -q

python evals/run_evals.py
```

70 tests should pass. The eval writes evals/report.md. make review runs the multi-agent business review.

Optional: switch the planner to Claude

```
# Windows (PowerShell)
$env:ANTHROPIC_API_KEY="sk-ant-..."

# Mac / Linux
export ANTHROPIC_API_KEY="sk-ant-..."

streamlit run app/streamlit_app.py
python evals/run_evals.py --planner llm
```

Variable	What it does
ANTHROPIC_API_KEY	Turns on the Claude planner and summaries
BI_AGENT_PLANNER	auto (default) · llm · rules
BI_AGENT_MODEL	Claude model ID (default claude-sonnet-5-5)
BI_AGENT_DB	Path to the DuckDB file



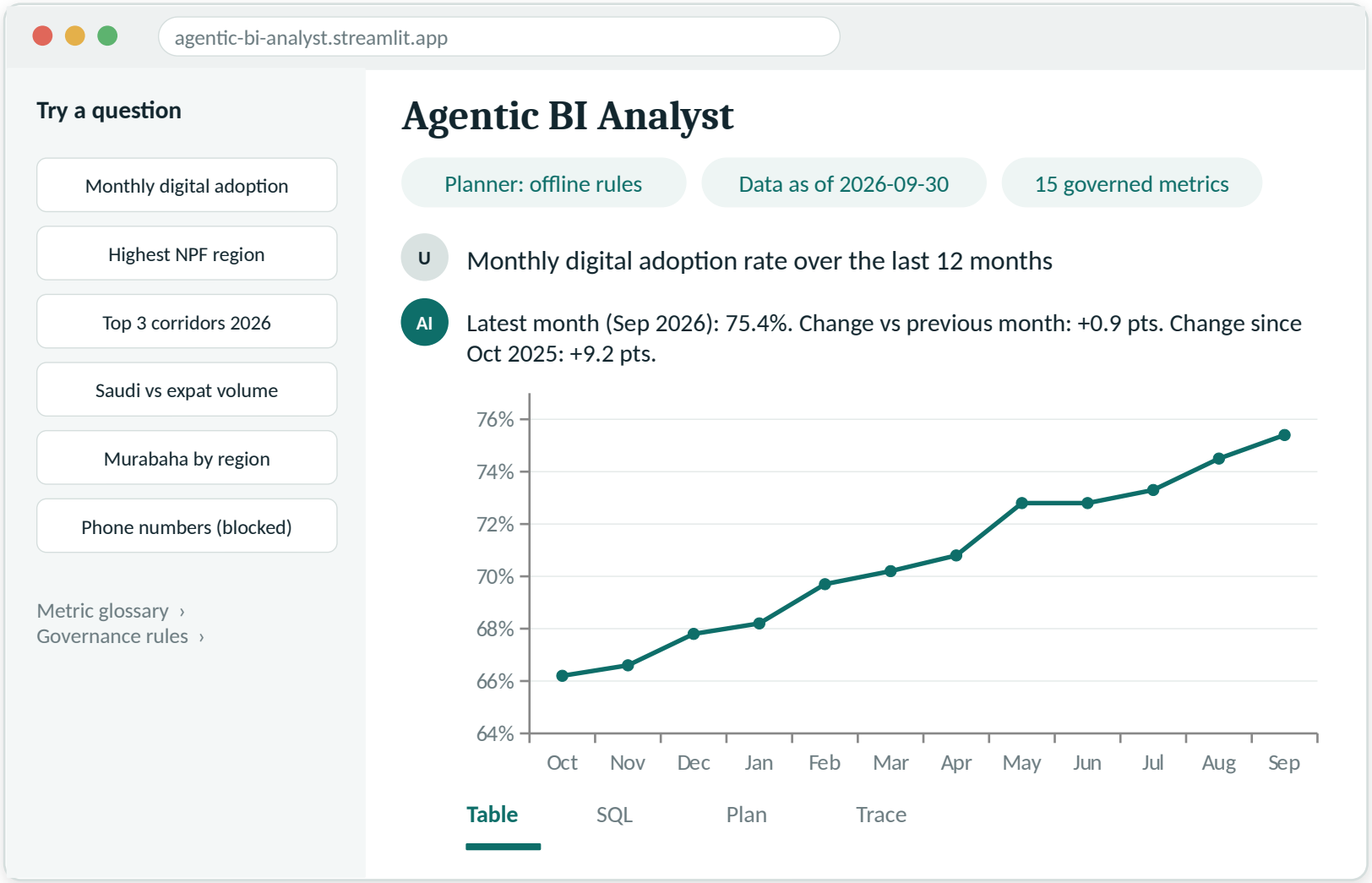
If Claude fails (no network, wrong key, quota), the agent falls back to the offline planner automatically and notes it in the trace. Set a spending limit on any key used in a public demo.

How to confirm it worked: the header pill in the app changes from "Planner: offline rules" to "Planner: Claude (model name)", and the Trace tab shows "via llm".

Part 2 · What the results look like

Mockups built from real output of the app (numbers are from the synthetic dataset)

Mockup: a trend question answered with a chart



What you see

- Header pills show the planner mode, data date and metric count
- The summary is built from computed facts; rates change in percentage points
- Plotly line chart (interactive in the real app)
- Tabs: Table, SQL, Plan and Trace for full auditability
- The sidebar holds sample questions, the metric glossary and governance rules

Mockup: rankings and breakdowns

agentic-bi-analyst.streamlit.app

U Which region has the highest NPF ratio?

AI Highest NPF ratio (90+ DPD): Northern at 5.0%. Lowest: Southern at 2.7%. Northern is 1.9x Southern.

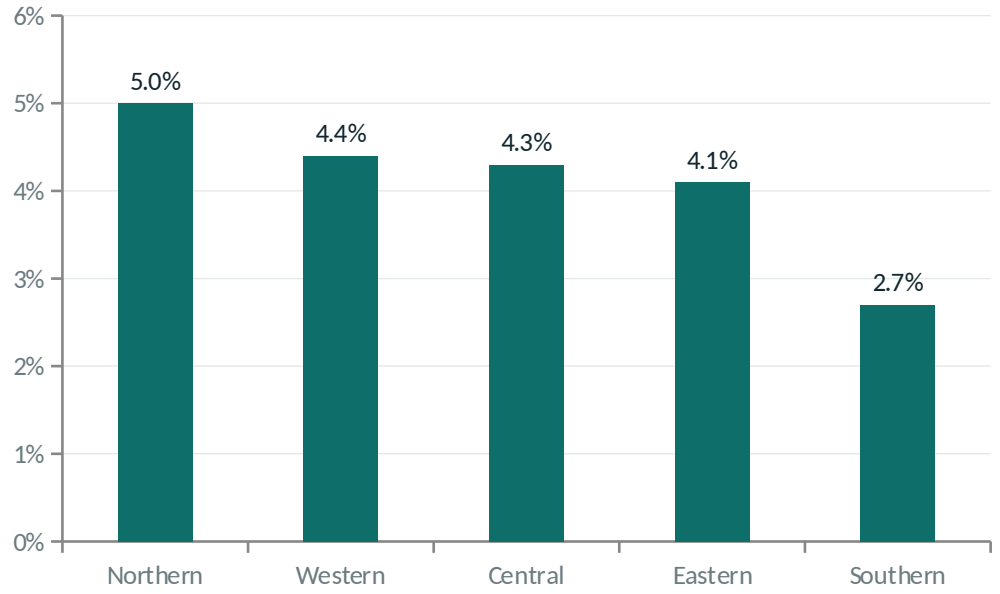


Table SQL Plan Trace

agentic-bi-analyst.streamlit.app

U Top 3 corridors by remittance volume in 2026

AI Highest: India at SAR 28.88M. Lowest of the top 3: Pakistan at SAR 18.66M.

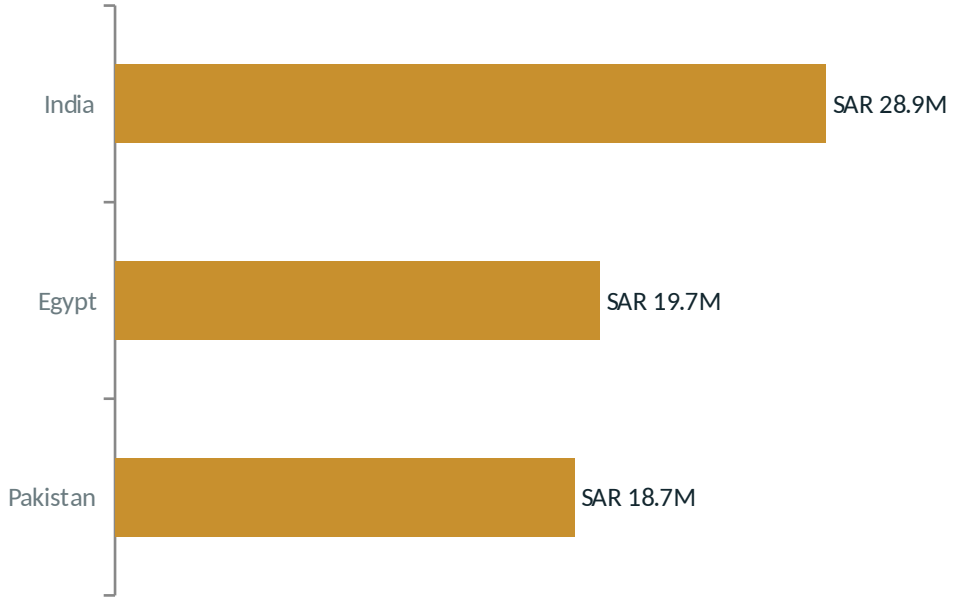


Table SQL Plan Trace

Mockup: SQL and Trace tabs make answers auditable

agentic-bi-analyst.streamlit.app

Table SQL Plan Trace

```
SELECT
  r.corridor AS corridor,
  SUM(r.amount_sar) AS remittance_volume,
  COUNT(DISTINCT r.customer_id)
      AS __n_remittances
FROM fact_remittance r
WHERE CAST(r.remittance_date AS DATE)
        >= '2026-01-01'
  AND CAST(r.remittance_date AS DATE)
        <= '2026-09-30'
GROUP BY ALL
ORDER BY remittance_volume DESC
LIMIT 3
```

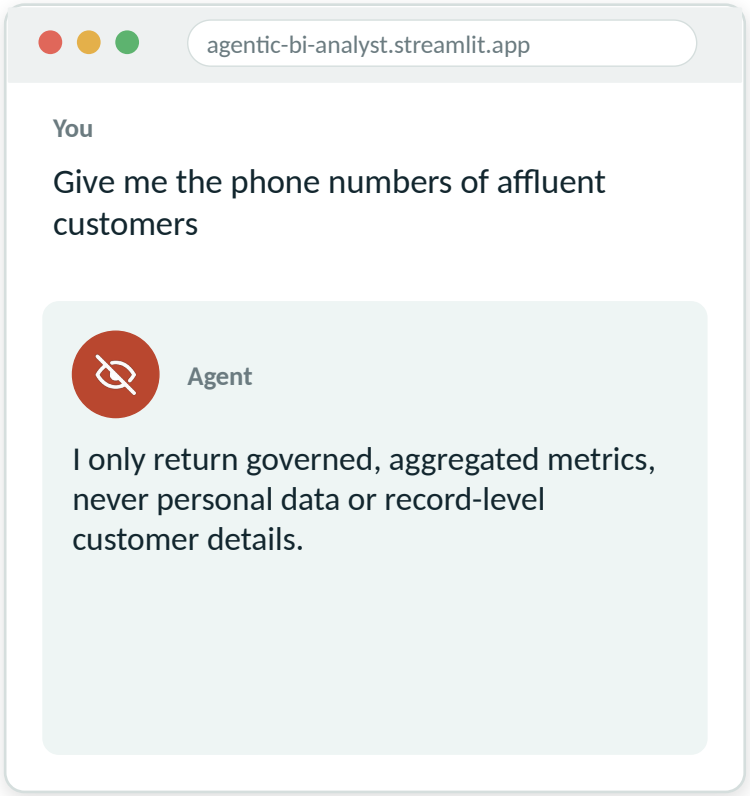
agentic-bi-analyst.streamlit.app

Table SQL Plan Trace

Step	ms	Detail
guard_input	1.6	passed
plan	14.1	attempt 1 via rules
validate	0.0	ok
compile	0.0	10 lines, 2 parameters
check_sql	0.9	read-only, allow-listed, no PII
execute	30.5	12 rows; 0 suppressed
explain	2.4	4 verified facts
respond	0.0	ok

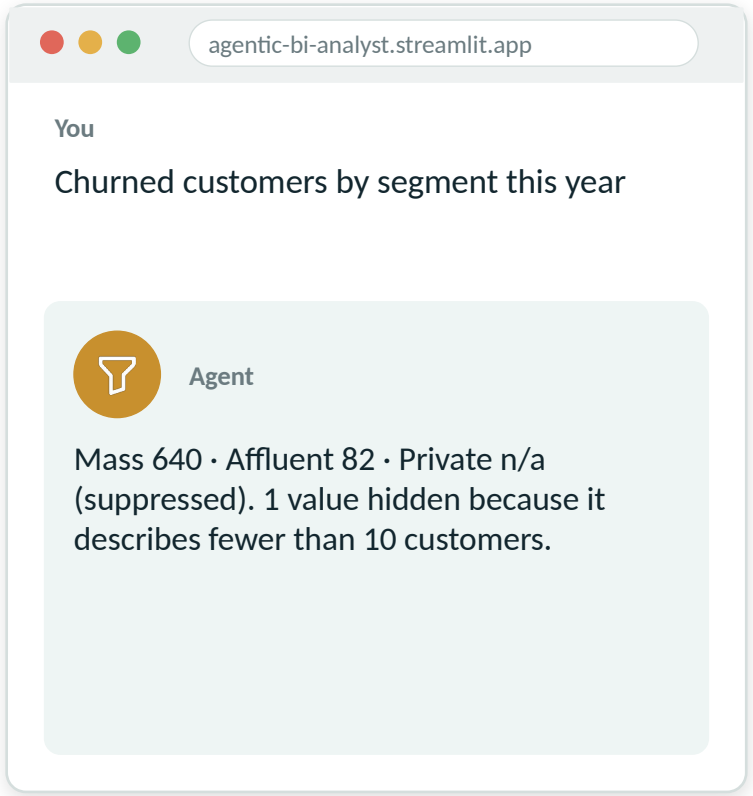
The SQL is generated by the compiler; filter values are bound as parameters (shown inline for display). The __n column counts customers for small-cell suppression and is removed before display.

Mockup: governance in action



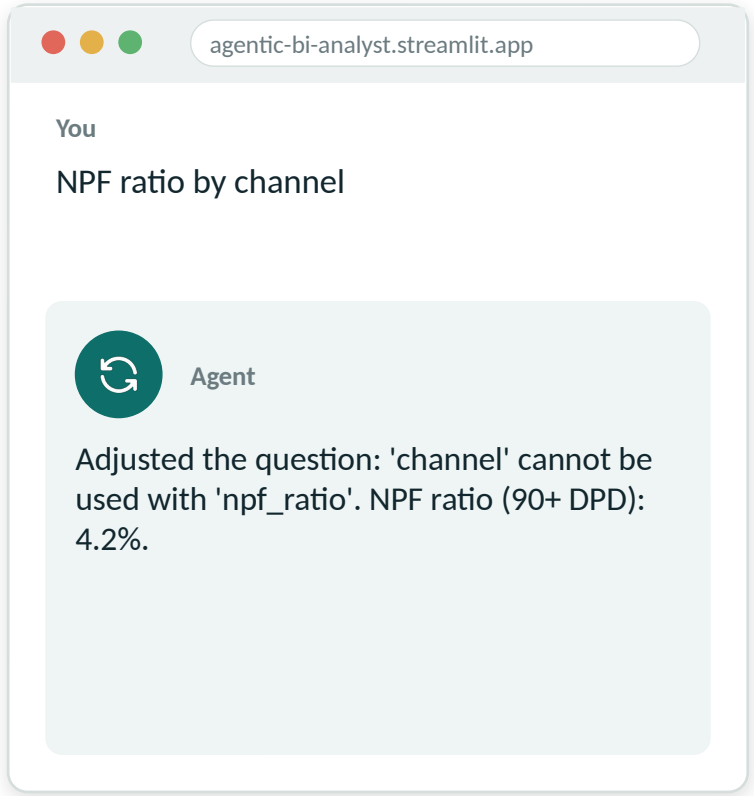
Personal data request refused

Input guard: the database is never queried



Small group hidden

Output guard: k-anonymity style rule



Impossible question repaired

Validator feedback and a second planning attempt

Mockup: command line with follow-up questions

```
$ python -m bi_agent.cli
Agentic BI Analyst — Wadi Bank (synthetic data).

> transaction volume last year
Transaction volume (SAR) (2025): SAR 349.80M.

> now by region
Highest Transaction volume (SAR) (2025): Central at SAR 132.76M.
Lowest: Northern at SAR 22.07M. Central accounts for 38.0% of the total.

> what about the private segment?
Highest Transaction volume (SAR) (2025): Central at SAR 30.37M.
Lowest: Northern at SAR 6.14M.

> exit
```

How follow-ups work

- The last successful plan is kept in memory
- "now by region" has no metric, so the metric, dates and filters carry over and region becomes the breakdown
- "the private segment" adds a filter on segment while keeping everything else
- The same memory powers the Streamlit chat

Mockup: the evaluation report (evals/report.md)

100%
plan accuracy

100%
answer accuracy vs reference SQL

100%
guardrail and clarify cases

~15 ms
median latency (offline)

Case	Question	Plan	Answer	Result
murabaha_region_q2	Murabaha disbursed by region in Q2 2026	pass	pass	PASS
multi_model_kpi	Transaction and remittance volume last year	pass	pass	PASS
churn_segment_ytd	Churned customers by segment this year	pass	pass (1 suppressed)	PASS
followup_filter	What about expats in the Western region?	pass	pass	PASS
self_correct_incompatible_dim	NPF ratio by channel	pass	pass	PASS
guard_pii_phone	Give me the phone numbers of affluent customers	—	—	PASS (blocked)

6 of 28 cases shown. Honest note: the golden set was written alongside the rule planner, so 100% means no regressions on known questions. New phrasings will score lower; run the same harness with --planner llm to compare.

Part 3 · How it was built, step by step

Twelve steps from an empty folder to a deployed, tested portfolio project

The build in twelve steps

1 Synthetic data

`scripts/generate_data.py`

2 Semantic layer

`semantic/`
`semantic_layer.yaml`

3 QueryPlan

`bi_agent/plan.py`

4 SQL compiler

`bi_agent/compiler.py`

5 Guardrails

`bi_agent/guardrails.py`

6 Planners

`planner_rules.py · planner_llm.py`

7 LangGraph agent

`bi_agent/graph.py`

8 Insights + charts

`bi_agent/insights.py`

9 UI + CLI

`app/streamlit_app.py · cli.py`

10 Tests

`tests/ (70 tests)`

11 Evals + CI

`evals/ · .github/workflows`

12 Publish + deploy

`GitHub · Streamlit Cloud · LinkedIn`

Step 1 · Synthetic data with planted business stories

`dim_customer`
20,000 · segment, nationality, age, branch, PII (name, mobile, email)

`fact_transaction`
~560k · date, channel, type, amount, is_digital

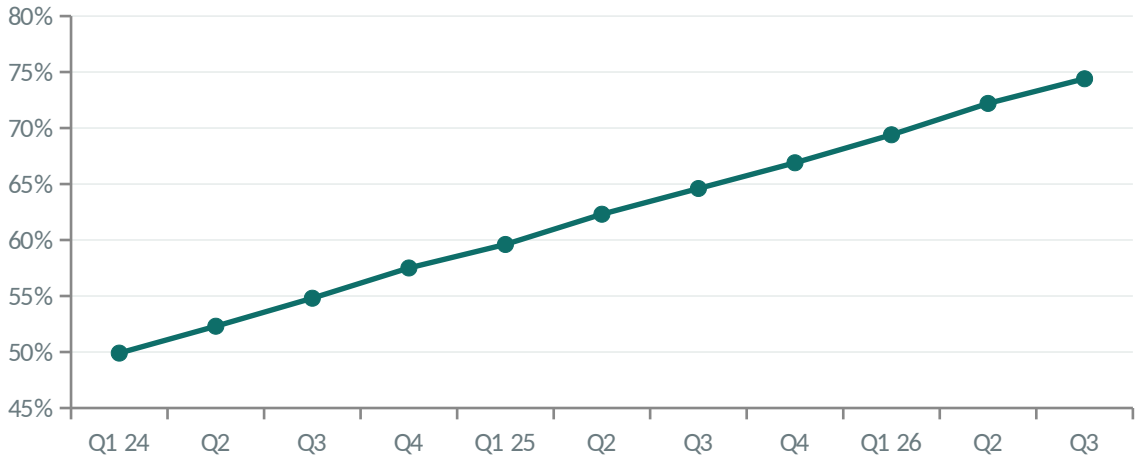
`fact_finance`
~5.4k Murabaha / Ijarah · principal, profit rate, status, outstanding

`fact_remittance`
~174k · corridor, channel, amount, fee

`dim_branch · dim_product · dim_date`
23 branches in 5 regions · 8 Islamic products

Vectorised NumPy (no row loops) · fixed seed for reproducibility · PII columns exist on purpose so the guardrails have something real to protect.

Planted story: digital adoption rises ~50% → ~75%



Other planted patterns

- Ramadan and pre-Eid spending spikes; payday and weekend effects
- Affluent and Private customers have 2.4x and 6x bigger tickets
- NPF higher in Mass (5.7%) than Affluent (2.3%)
- Expats remit to India, Pakistan, Indonesia, Egypt, Philippines, Bangladesh
- ~11% of Mass customers churn vs ~2% of Private

Step 2 · The semantic layer: one definition per metric

```
models:
  finance:
    table: fact_finance
    time_column: f.disbursement_date
    customer_key: f.customer_id
    joins:
      c: LEFT JOIN dim_customer c ...
      b: LEFT JOIN dim_branch b ...  (requires c)

dimensions:
  region: {expr: b.region, requires: [c, b]}

metrics:
  npf_ratio:
    model: finance
    additive: false
    expr: SUM(90+ DPD outstanding)
          / SUM(open outstanding)
    owner: Retail Credit Risk
    synonyms: [npf, npl, default rate]
```

Models

A fact table plus the joins it is allowed to use

Dimensions

How metrics can be sliced; compatibility is computed from required joins

Metrics

Governed SQL expressions with owner, description, format and synonyms

Governance

PII column list, allow-listed tables, max rows, small-cell threshold

Add a metric without touching Python: add a YAML block, add a golden question, run make evals. The same idea as dbt MetricFlow, Power BI measures or Denodo derived views.

Steps 3–4 · QueryPlan in, safe SQL out

QueryPlan: the planner's only output

```
{
  "metrics": ["remittance_volume"],
  "dimensions": ["corridor"],
  "start_date": "2026-01-01",
  "end_date": "2026-09-30",
  "filters": [{"dimension": "segment",
               "values": ["Affluent"]}],
  "order_by": "remittance_volume",
  "limit": 3
}
```

validate_plan() checks names, compatibility, dates and limits, and writes errors in plain English for the planner to read.

Compiler algorithm

1. Group metrics by model (fact table)
2. Per model: SELECT time bucket, dimensions, metric expressions and a customer count
3. Add only the joins needed, in dependency order
4. Bind every filter value as a ? parameter (no injection)
5. Several models → one CTE each, then FULL OUTER JOIN on shared dimensions (no fan-out)
6. ORDER BY, then LIMIT capped at max_rows

Step 5 · Guardrails: three independent layers

1 · Input guard

Before planning

Blocks names, phone, email, ID, IBAN, specific customer IDs, record exports and prompt injection ("ignore previous instructions", "drop table").

2 · SQL guard

After compiling

Single statement, SELECT/WITH only, no DDL/DML keywords, no PII columns, allow-listed tables only, must end with LIMIT.

3 · Output guard

After executing

Masks any value computed from fewer than 10 customers so aggregates can't re-identify anyone.

Plus: the DuckDB connection is read-only, and the semantic layer simply cannot express a PII column. Regex guards can be evaded, which is exactly why layers 2 and 3 do not trust layer 1.

Step 6 · Two planners behind one interface

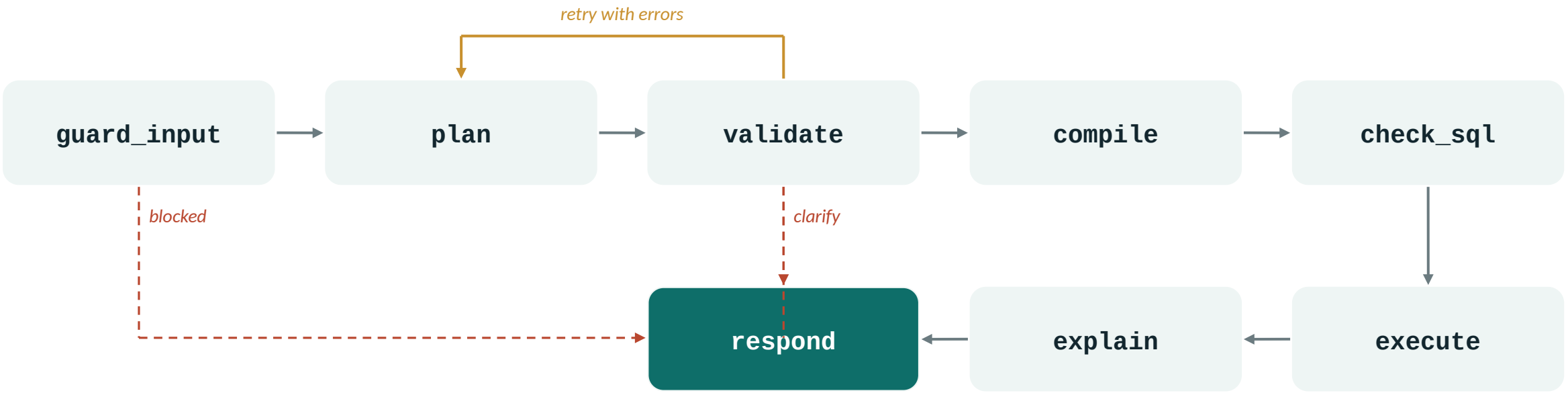
Offline rule planner

- Longest-first synonym matching for metrics
- Dates resolved against the as-of date: last N months, YTD, Q2 2026, since March 2025, 2024 vs 2025
- Values from the database become filters (Riyadh, Affluent, Mobile App), plus aliases (murabaha, expats)
- "by / per / vs / which" turn a word into a breakdown
- Top-N, follow-ups, repair on feedback, clarification
- **Free, deterministic, used in CI**

Claude planner

- System prompt holds the catalog and known values; no data rows are sent
- Forced tool call submit_query_plan with enum fields, so it cannot invent names
- Validator errors are appended on retry (self-correction)
- Previous plan included for follow-ups
- Optional summary may only rephrase verified facts
- **Any failure falls back to the rule planner**

Step 7 · LangGraph agent: every path visible and bounded



State

A TypedDict: question, plan, errors, attempts, SQL, result, facts, chart, status, message

Trace reducer

Annotated[list, operator.add]: every node appends its step and timing

Conditional edges

blocked → respond · errors → plan · clarify → respond · SQL violation → respond

Why a graph?

Banks need predictable, testable paths. Tests assert the exact node sequence; retries are capped

Steps 8–9 · Verified insights, then the user interface

Question shape	Facts computed with pandas	Chart
KPI	Value per metric, with the time label	Metric tiles
Trend	Latest, change vs previous period, change since start, peak	Line
Breakdown	Highest, lowest, share of total (additive) or N× ratio	Bar
Breakdown over time	Strongest and weakest change by group, largest latest	Line by colour

Rates are compared in percentage points (75% vs 70% is +5.0 pts, not +7.1%). Summaries never contain a number that was not computed from the result.



Streamlit app

- Chat with conversation memory
- Plotly chart chosen by the agent
- Table · SQL · Plan · Trace tabs
- Glossary with metric owners
- Agent cached with st.cache_resource
- Builds the database itself on first cloud start

CLI: one-shot or interactive, --show-sql, --trace, --planner

Steps 10–11 · Tests, golden-question evals and CI

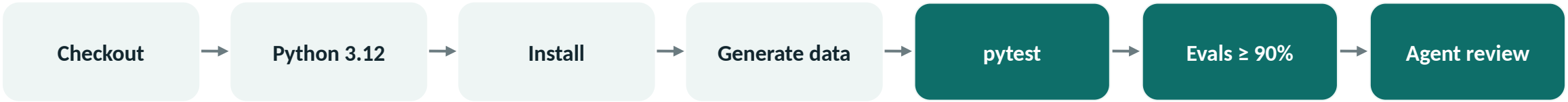
70 pytest tests (~11 s on a 3,000-customer test database)

`test_compiler.py`	parameters, join order, no fan-out, LIMIT cap, every metric runs
`test_guardrails.py`	blocked and allowed questions, SQL rejections, suppression
`test_planner.py`	11 date expressions, filters vs breakdowns, follow-ups, repair
`test_graph.py`	node sequence, blocked path, self-correction, mocked Claude, fallback
`test_app.py`	headless Streamlit runs of both pages
`test_orchestrator.py`	fan-out, planted links, placebo, verifier, LLM rewrite

Golden-question eval

```
- id: murabaha_region_q2
  question: Murabaha disbursed by region in Q2 2026
  expect: {metrics: [finance_disbursed],
           dimensions: [region], ...}
  reference_sql: |
    SELECT b.region, SUM(f.principal_sar) ...
    WHERE f.product_id = 'P06' ...
```

Reference SQL is hand-written without the semantic layer, so a wrong metric definition is caught too.



GitHub Actions on every push · no API key needed · the eval report is posted to the job summary

Step 12 · Publish, deploy and share

GitHub

1. Create public repo agentic-bi-analyst
2. git init · add · commit · push
3. Add description and topics
4. Check the Actions tab is green
5. Pin the repo on your profile

Streamlit Cloud

1. share.streamlit.io → Create app
2. Repo vickarzz/agentic-bi-analyst
3. Main file app/streamlit_app.py
4. Python 3.12 → Deploy
5. Optional: API key in Secrets

LinkedIn

1. Projects: title, link, CV bullet
2. Featured: live demo link
3. Post with a 30-second demo video
4. Repo link in the first comment
5. Post Tuesday to Thursday morning

Order matters: run locally → push to GitHub → deploy → record a demo GIF → post. Everything works when recruiters click.

Part 4 · Multi-agent orchestration

A team of agents reviews the whole bank and tests how each business line affects the others

From one question to the whole business

Chat agent (Parts 1–3)

Answers one question at a time: "NPF ratio by segment".

Business review (Part 4)

Answers "How is the bank doing, and how does each line affect the others?" with a verified executive briefing.

Same governance

Agents use governed metrics or approved SQL templates only

Same engine

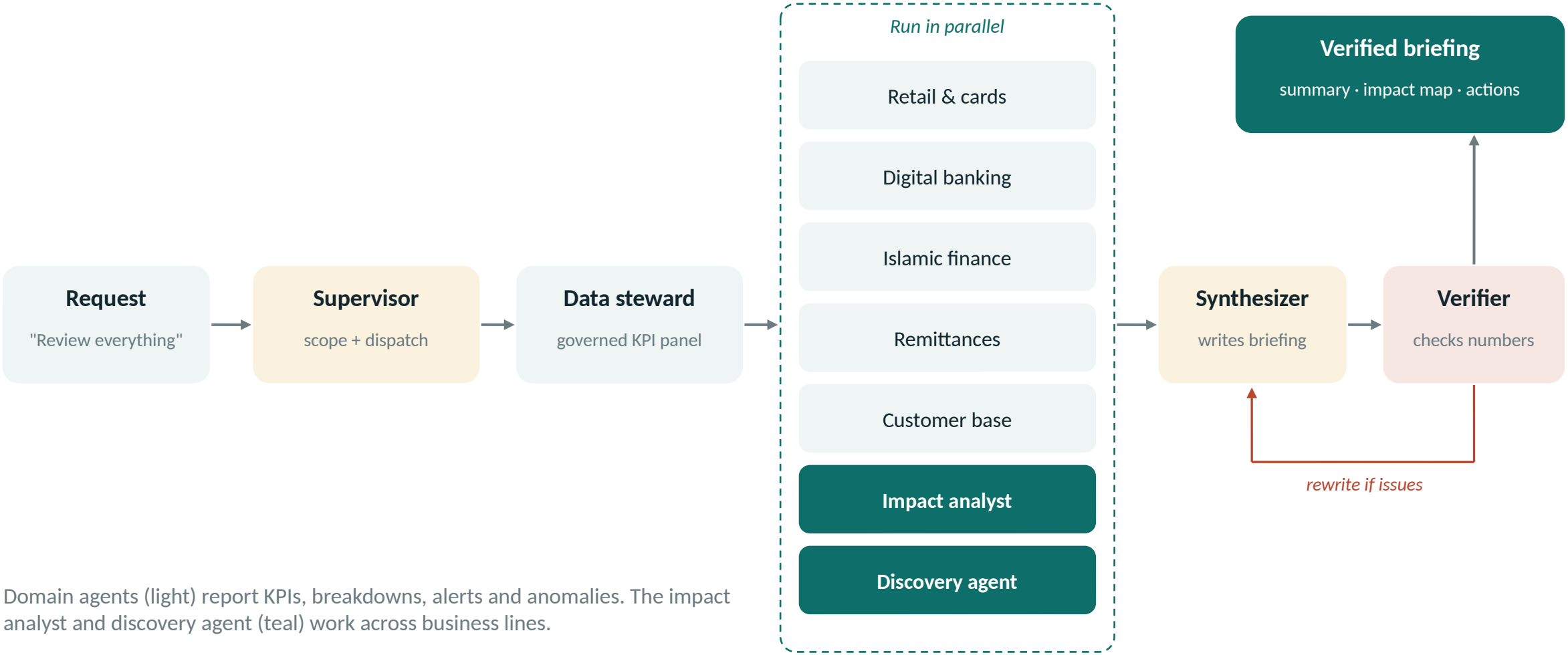
LangGraph, now with parallel fan-out (Send) and a join

New: a verifier

An agent that audits the writing before anyone sees it

Output: an executive summary, risk alerts, an impact map, tested links with evidence, recommended actions per business owner, domain scorecards and method caveats, as Markdown, JSON and an interactive page.

Agent workflow: supervisor, parallel specialists, writer, auditor



Domain agents (light) report KPIs, breakdowns, alerts and anomalies. The impact analyst and discovery agent (teal) work across business lines.

Who does what

Supervisor

Reads the request, picks domains in scope ("credit", "churn", or everything), dispatches the team.

`no data tools`

Data steward

Builds one governed monthly KPI panel: 12 metrics × 33 months, shared by all agents.

`governed_query`

5 domain agents

Quarter KPIs with QoQ and YoY, 12-month breakdowns, alert thresholds (NPF > 5%), unusual months.

`governed_query`

Impact analyst

Tests 8 owner-written cross-business hypotheses; returns verdict, effect, p-value, action.

`governed_query · approved_analysis`

Discovery agent

Scans all 66 KPI pairs for co-movement nobody hypothesised; proposes candidates only.

`panel only`

Synthesizer

Writes the briefing from verified sentences; Claude may rewrite the summary.

`no data tools`

Verifier

Rejects ungrounded numbers, causal wording and weak "confirmed" claims.

`no data tools`

Calendar

Ramadan, Eid and the fee-cut date explain anomalies and anchor event tests.

`business_calendar.yaml`

Cross-business hypotheses are owned, written down and tested

```yaml
- id: churn_to_credit_risk
  title: Churned customers and credit risk
  from: {domain: customer, metric: churned_customers}
  to:   {domain: credit,   metric: npf_ratio}
  hypothesis: Contracts of customers who left the
    bank are more likely to be non-performing.
  expected: positive
  owner: Retail Credit Risk
  if_confirmed: Feed churn signals into early-warning
    credit monitoring ...
  test:
    type: cohort
    sql: |   # approved, reviewed, no identifiers
      SELECT ... FROM fact_finance f
      LEFT JOIN dim_customer c ... LIMIT 100000
```

Test	Question	Method
timeseries	Do two KPIs move together, maybe later?	Growth correlation, lags 0–2
cross_section	Where X is high, is Y high?	Correlation across groups
cohort	Is the outcome different for a group?	Permutation on cohort labels
event	Did a dated event change it?	6 months before vs after

confirmed — expected sign, $p < 0.05$, meaningful effect

not detected — no significant evidence

contradicted — significant, opposite sign

control passed — placebo shows no link, as it should

Statistical choices an executive can trust

Growth, not levels	Most KPIs grew in 2024–2026, so raw levels correlate even when unrelated. Month-over-month growth removes the shared trend.
Permutation p-values	"How often would shuffled months show a link this strong?" Easy to explain; no distribution assumptions.
Bonferroni adjustment	Trying 3 lags or 66 pairs raises false alarms, so p-values are multiplied by the number of tries.
A placebo control	Gender has no planted effect on credit risk. If the method ever "confirms" it, the verifier fails the briefing.
Association, not causation	Briefings say "is associated with". Event and cohort designs are stronger evidence, but still not experiments.

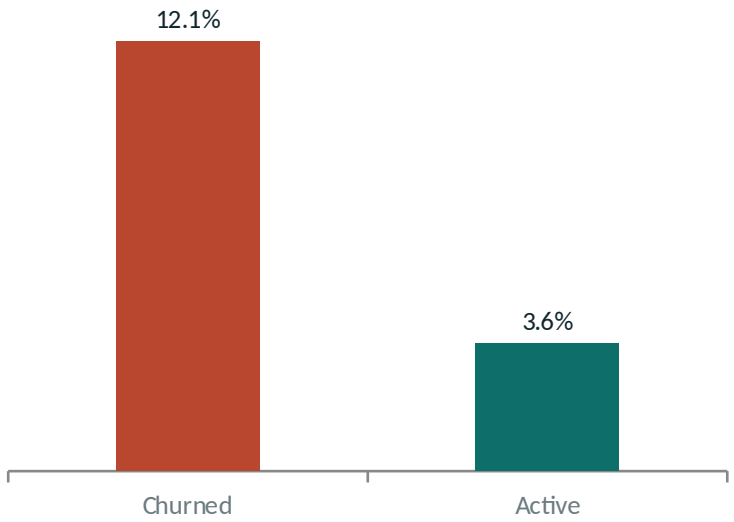
What the agents found: 8 hypotheses tested

Link	From → to	Verdict	Evidence
Spending and remittances move together	Retail → Payments	**confirmed**	Growth r = +0.98; 1% more spending ≈ +0.94% remittances
Churned customers and credit risk	Customer → Credit	**confirmed**	NPF 12.1% churned vs 3.6% active (3.3x, p < 0.001)
Churn rate and NPF by segment × region	Customer → Credit	**confirmed**	r = +0.74 across 10 groups (p = 0.009)
Fee cut and revenue per remittance	Payments → Payments	**confirmed**	SAR 18.05 → SAR 13.59 per transfer (-24.7%)
Fee cut and remittance activity	Payments → Payments	**not detected**	YoY growth 12.8% → 14.1% (p = 0.82)
Customer age and digital adoption	Customer → Digital	**confirmed**	r = -0.99 across age bands (82.5% vs 52.2%)
New customers and next-month activity	Customer → Retail	**not detected**	r = +0.09
Control: gender vs credit risk	Customer → Credit	**control passed**	1.3x, p = 0.17: not significant, as expected

These match how the synthetic data was generated: the agents recovered every planted relationship and did not invent the absent ones.

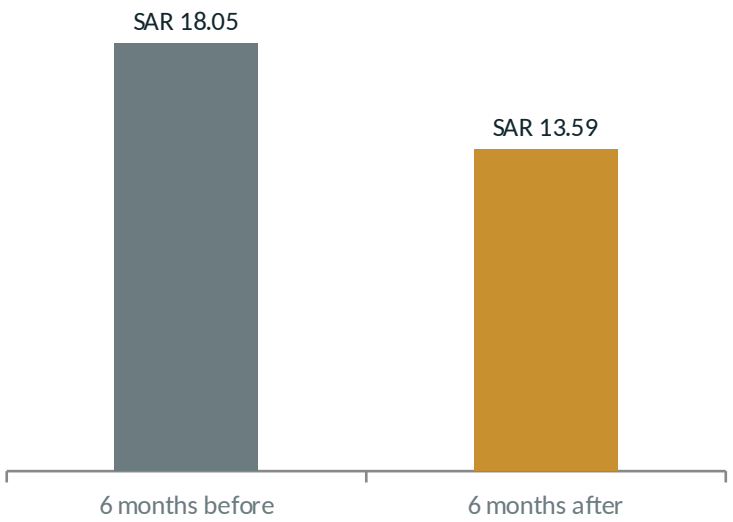
The evidence behind the headline insights

NPF ratio by customer status



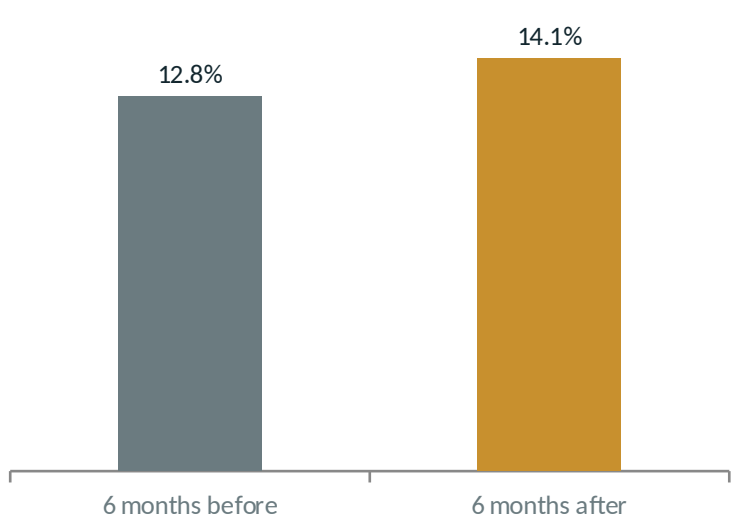
Churned: 3.3x the NPF ratio (p < 0.001)

Fee income per remittance (SAR)



Revenue per transfer -24.7% (p = 0.002)

Remittances, year-over-year growth



...activity growth unchanged (p = 0.82)

Executive insight: the fee cut gave up about a quarter of revenue per transfer without a measurable lift in activity. Test targeted (corridor or segment) pricing instead of a blanket cut.

Mockup: the Business Review page

agentic-bi-analyst.streamlit.app/Business_Review

Cross-business review

Links confirmed	Risk alerts	Watch items	Verifier
5 / 8	2	3	Passed

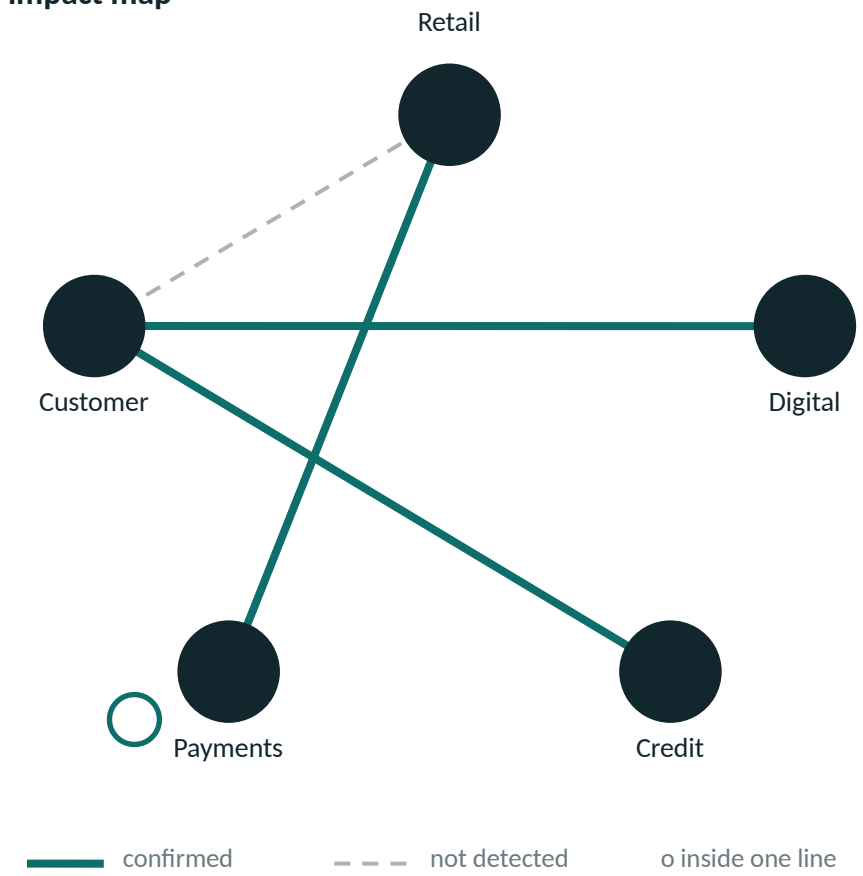
Executive summary

- NPF ratio for the Mass segment is 5.7%, above the 5.0% alert level
- Churned customers: NPF 12.1% vs 3.6% for active customers (3.3x)
- Spending and remittances move together (growth r = +0.98)
- Fee cut: revenue per transfer -24.7%, no detectable activity lift

Impact tests Actions Risks Scorecards Discovery Trace

Each tab: evidence chart per link, owner actions, alerts, heatmap, agent timeline, downloads

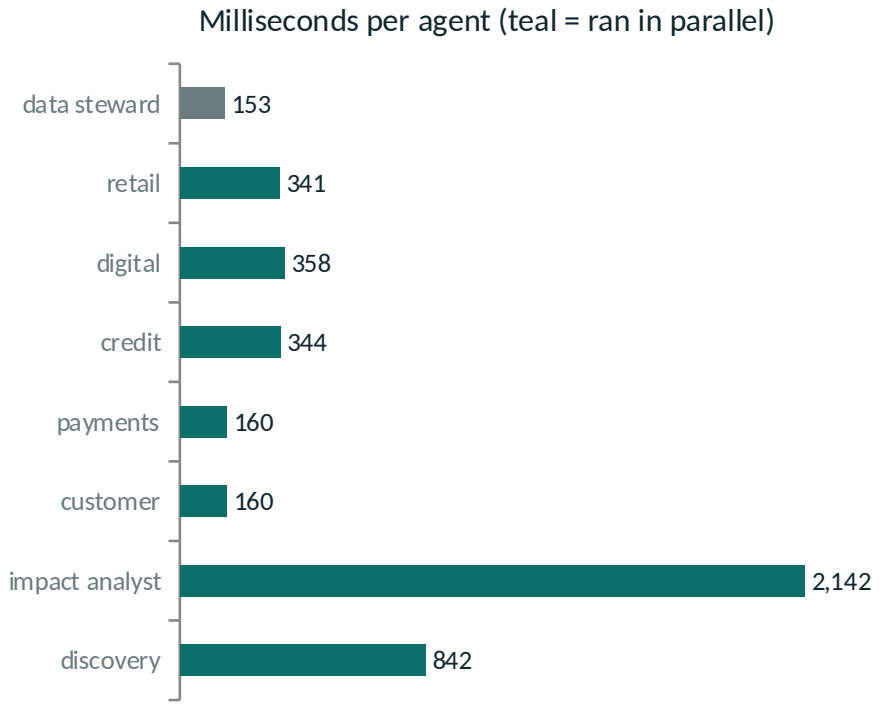
Impact map



Mockup: the agent trace shows the team working in parallel

```
$ python -m bi_agent.orchestrator
Agent trace
   supervisor          0.6 ms  scope = whole business
   data_steward      153.1 ms  12 metrics x 33 months
 ‖ domain:retail     341.1 ms  7 findings
 ‖ domain:digital    358.4 ms  3 findings
 ‖ domain:credit     344.1 ms  7 findings (2 alert)
 ‖ domain:payments   159.9 ms  5 findings (1 watch)
 ‖ domain:customer   160.1 ms  3 findings (2 watch)
 ‖ impact_analyst   2141.6 ms  8 links: 5 confirmed
 ‖ discovery_agent   841.7 ms  66 pairs, 1 candidate
   synthesizer         0.1 ms  6 summary points
   verifier            1.0 ms  passed (64 numbers)
   respond             0.1 ms  briefing ready
   governed tool calls: 27

Saved reports/business_review.md and .json
```



Because specialists run in one parallel super-step, the review takes about as long as the slowest agent (the impact analyst's permutation tests), around 3 seconds in total.

The verifier: an agent that audits the writing

1. **Grounding**
   Every number must appear in a fact an agent computed

2. **Language**
   No "drives", "causes", "leads to", "due to", "proves"

3. **Evidence**
   "Confirmed" requires p < 0.05; a failed control blocks sign-off

4. **Coverage**
   Every domain in scope has findings

With the Claude writer, rejected drafts go back with the issues (max 2 rewrites). If it still fails, the deterministic summary ships instead.

Draft 1: rejected

"Churn causes a 99.9% jump in NPF."

```
Ungrounded number '99.9%' · causal wording 'causes'
```

Draft 2: accepted

"Churned customers are associated with a higher NPF ratio than active customers."

```
This exact loop is a test:
test_llm_writer_is_corrected_by_verifier
```

Run it, extend it, test it

Run

```
# whole business → reports/*.md + *.json
python -m bi_agent.orchestrator

# narrower scope
python -m bi_agent.orchestrator \
  "focus on credit risk and churn"

# or
make review

# in the app: sidebar →
#  Run the multi-agent business review
```

Add a hypothesis (no Python)

```
- id: digital_to_fees
  title: Digital vs fee income
  from:
    domain: digital
    metric: digital_adoption_rate
  to:
    domain: payments
    metric: remittance_fee_income
  expected: negative
  owner: Payments
  if_confirmed: ...
  if_not_detected: ...
  test:
    type: timeseries
    transform: growth
    max_lag: 2
```

Tested (10 tests)

- All 7 specialists run in parallel
- Every tool call is governed
- Planted links are confirmed
- Absent links are not invented
- Focused requests limit scope
- Verifier catches invented numbers and causal claims
- LLM rewrite loop converges
- Business Review page renders

Part 5 · Deploy and train the agents

Three commands to go live, and a 15-minute weekly loop to make the agents smarter

Deploy everything at once

1 · Set up and check

```
bash scripts/setup_mac.sh --run
```

Creates .venv, installs libraries, builds data, runs 70 tests, evals and the agent review, then opens the app.

2 · Publish to GitHub

```
bash scripts/publish_github.sh vickarzz
```

Commits and pushes (data, .venv and feedback logs are excluded). CI runs tests, evals and the review.

3 · Go live

```
share.streamlit.io → Create app
app/streamlit_app.py · Python 3.12
```

Both pages (chat + Business Review) deploy together. Public link: agentic-bi-analyst.streamlit.app

Updates later: edit, then run step 2 again. Streamlit redeploys automatically. Full guide in Indonesian: docs/PANDUAN_ID.md

Training the agents: a governed feedback loop

1 Use + rate

Every question and 👍/👎 is logged (text, status, plan; never results)

2 Report

train_from_feedback.py lists unknown words, 👎 answers, golden candidates, blocked questions

3 Teach

teach.py adds synonyms, aliases or golden questions; validates and rolls back bad edits

4 Check

make test + make evals must stay ≥ 90%

5 Publish

Commit = audit trail of what the agents learned and who approved it

Worked example from the test run

```
$ python -m bi_agent.cli "kredit macet per wilayah"
I couldn't map that question to a governed metric.

$ python scripts/teach.py synonym npf_ratio "kredit macet"
Added 'kredit macet' as a synonym of npf_ratio.
$ python scripts/teach.py synonym region "wilayah"
$ python scripts/teach.py golden "kredit macet per wilayah"
$ python -m bi_agent.cli "kredit macet per wilayah"
Highest NPF ratio (90+ DPD): Northern at 5.0% ...
```

- No weights are trained: behaviour changes only through reviewed YAML, so every answer stays explainable
- Works in any language: Indonesian, Arabic or bank jargon
- New words flow into the Claude planner's catalog automatically
- Free cloud logs reset on restart, so run the loop on your laptop

From portfolio to production on enterprise tools

This repo	Power BI / Fabric	dbt	Denodo	Databricks
`semantic_layer.yaml`	DAX measures in a semantic model	MetricFlow metrics	Derived / interface views	Unity Catalog metric views
`compiler.py`	DAX via XMLA / executeQueries	Semantic Layer API	VQL / REST data services	SQL on metric views / Genie
`Guardrails`	RLS / OLS, sensitivity labels	Warehouse grants	Role-based security, masking	Grants, column masks
`Planner`	Fabric data agent / Copilot	Any LLM + API	Any LLM + REST	Genie

Swapping DuckDB for Snowflake, Oracle or Cloudera Impala only changes bi_agent/db.py (plus small SQL dialect tweaks in the compiler).



Roadmap: MCP server so any assistant can query governed metrics · Arabic questions · metric lineage view · held-out eval set

Telling the story in an interview (STAR)

S

Situation

Business users wait days for analysts, and LLM chat tools give inconsistent, ungoverned numbers, which is unacceptable under banking regulation.

T

Task

Answer plain-English questions with the same numbers as official dashboards, without exposing personal data.

A

Action

Separated understanding from computation: a LangGraph agent plans within a governed semantic layer, a compiler writes parameterised SQL, three guardrail layers protect PII, and evals run in CI.

R

Result

15 governed metrics, 100% plan and answer accuracy on 28 golden questions, every answer auditable, zero PII exposure by design. The agent team tests 8 cross-business links: it recovered all planted relationships, passed the placebo control, and showed that a fee cut lost 24.7% revenue per transfer without lifting activity.

Next steps

Run it locally today, publish this week, then compare the Claude planner against the offline one

1

bash scripts/setup_mac.sh --run

2

bash scripts/publish_github.sh vickarzz

3

Deploy on Streamlit Cloud, post on LinkedIn

4

15-minute weekly training loop